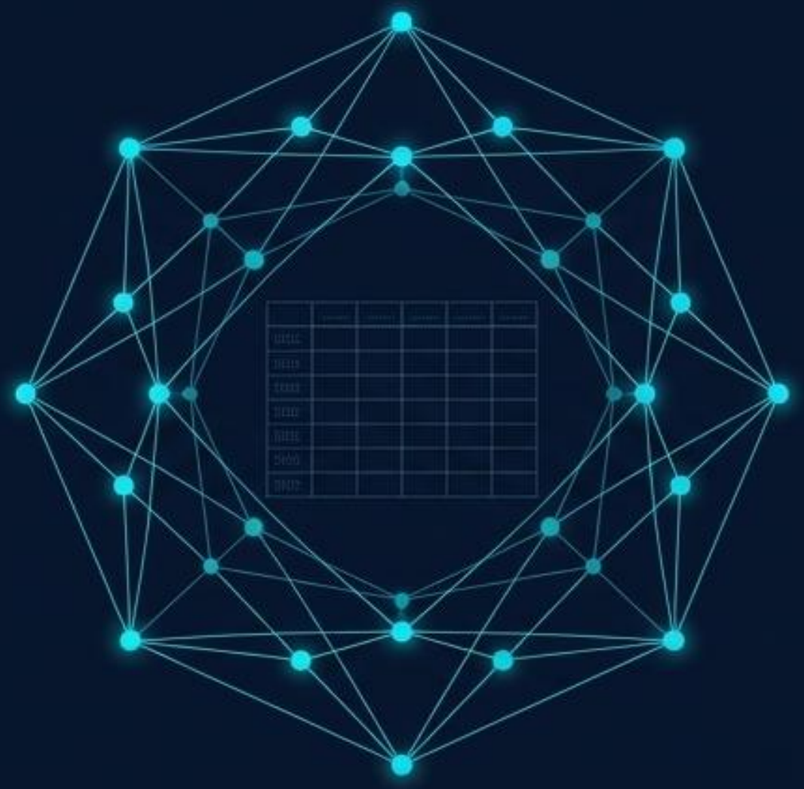


Pandas를 활용한 데이터 처리 및 연산

DataFrame 익히기 Part 1: 생성 및 추가/삭제

Data Engineering Foundation

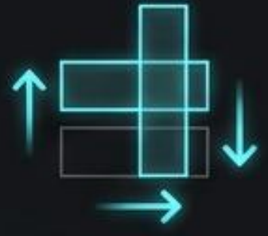
학습 목표 (Learning Objectives)

1. DataFrame 생성

2. 행과 열 추가/삭제

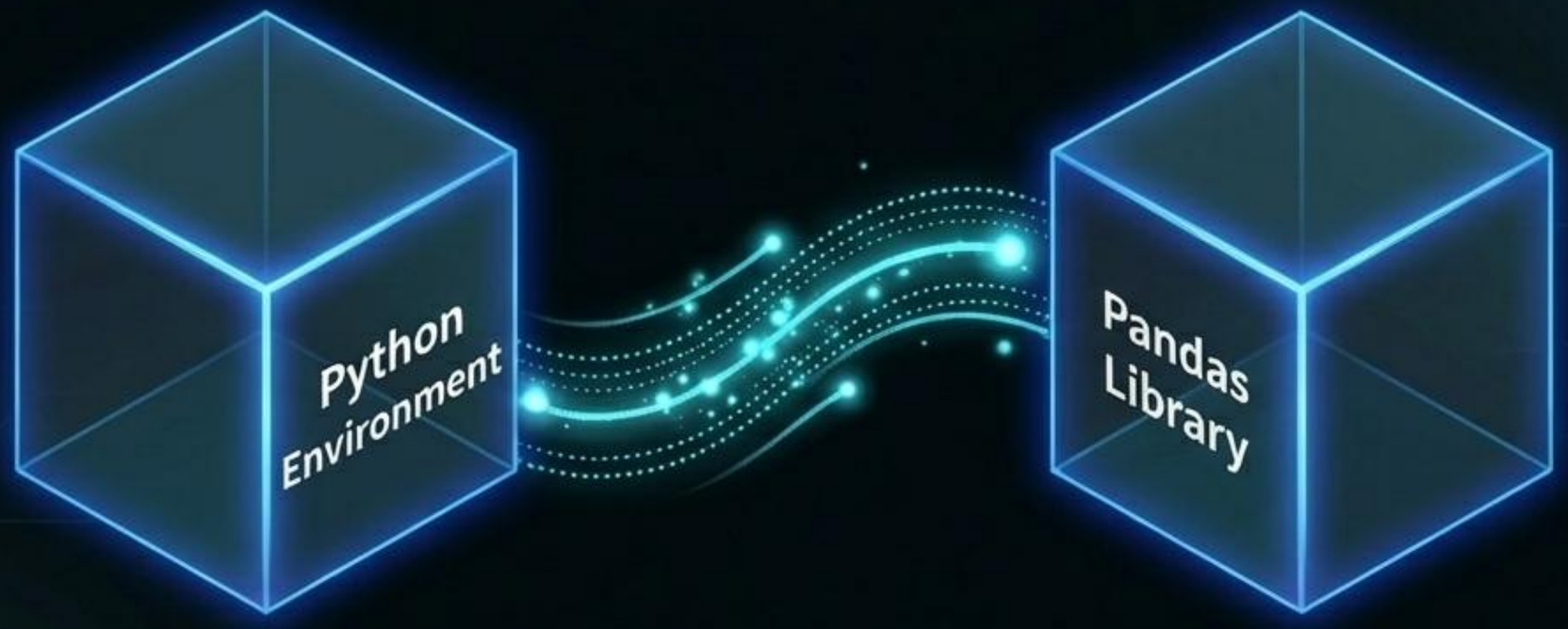
3. 행과 열, 원소 선택

4. 행과 열의 위치 변경



진행 순서 (Session Agenda)

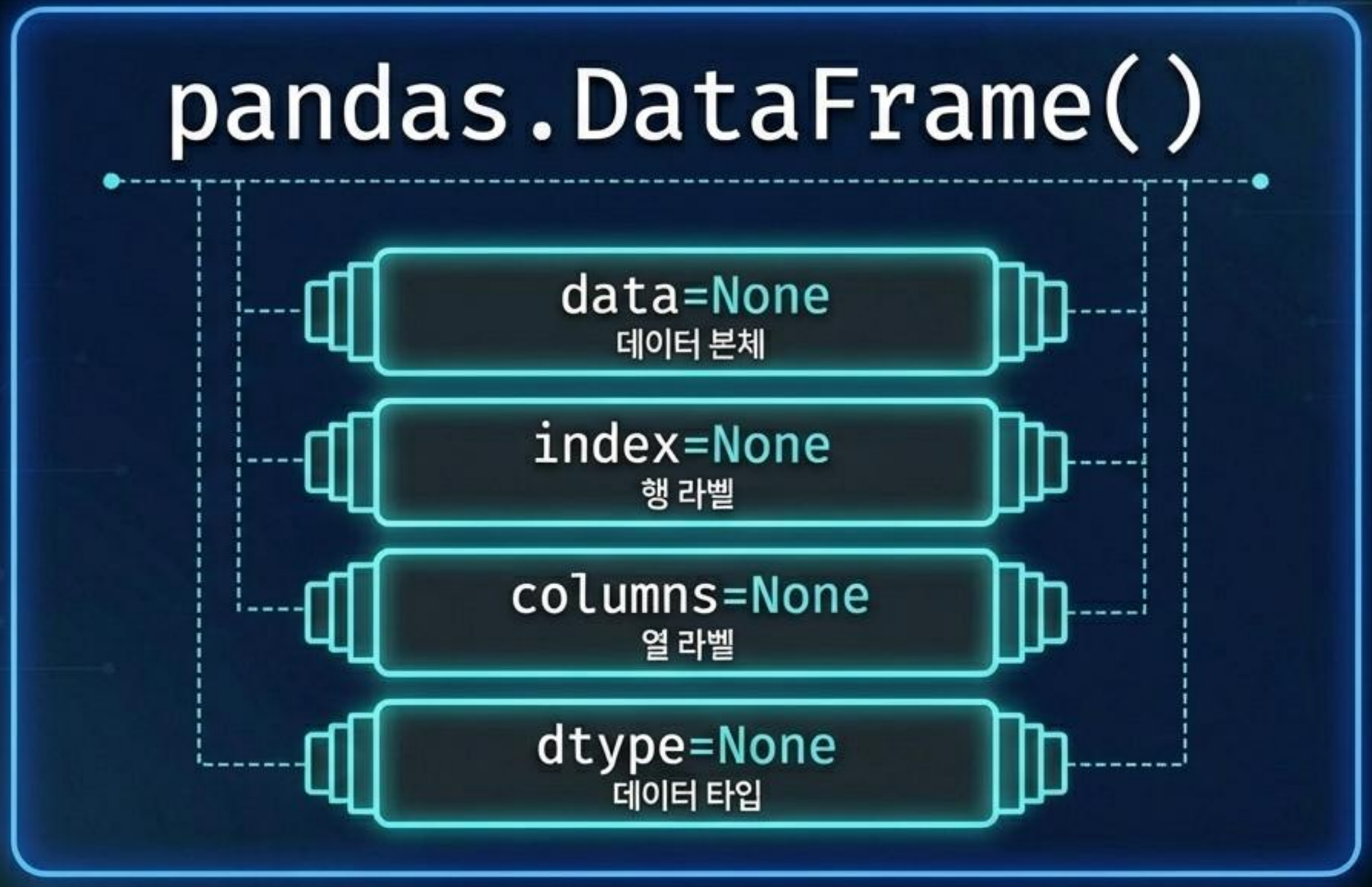
DataFrame 준비 (Initialization)



Pandas 라이브러리 호출 및 환경 세팅

```
> import pandas as pd
```

DataFrame 기본 형식 (Syntax Blueprint)



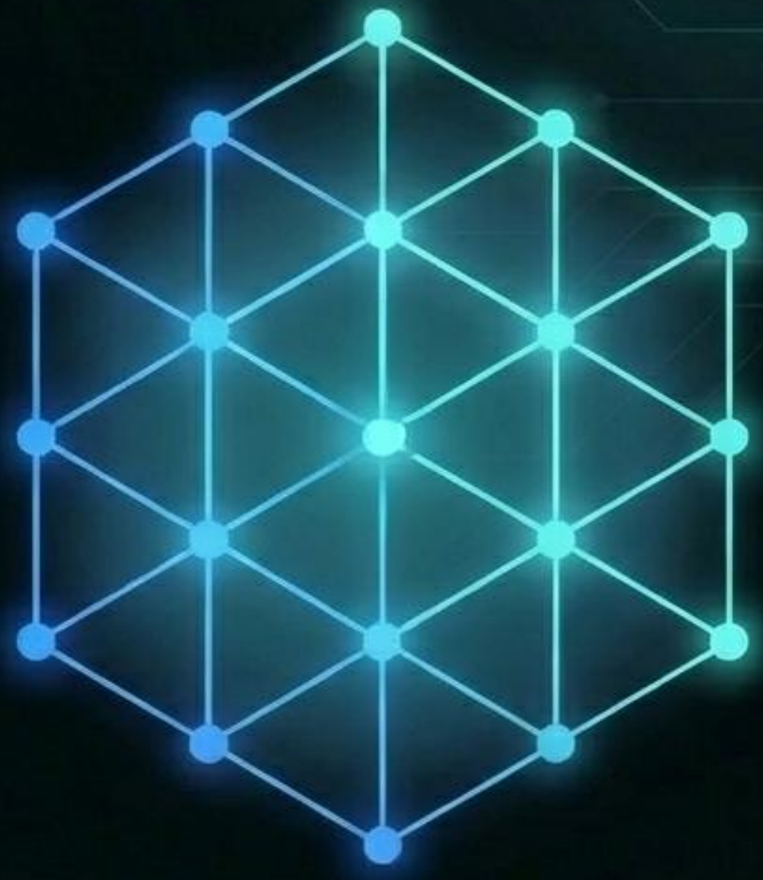
Pandas 데이터 타입 (dtype vs type)

dtype (Pandas)	type (Python)
int64	integer
float64	decimal
object	string
bool	boolean
datetime64	date & time
category	category

생성 기법 1: 칼럼명 지정

데이터 값과 열(Column) 이름만 명시적 주입

```
df = pd.DataFrame(
    data=[[10,40],[20,50],[30,60]],
    columns=['c1','c2']
)
```



결과 출력 구조 (Output Visualization 1)

(지정)

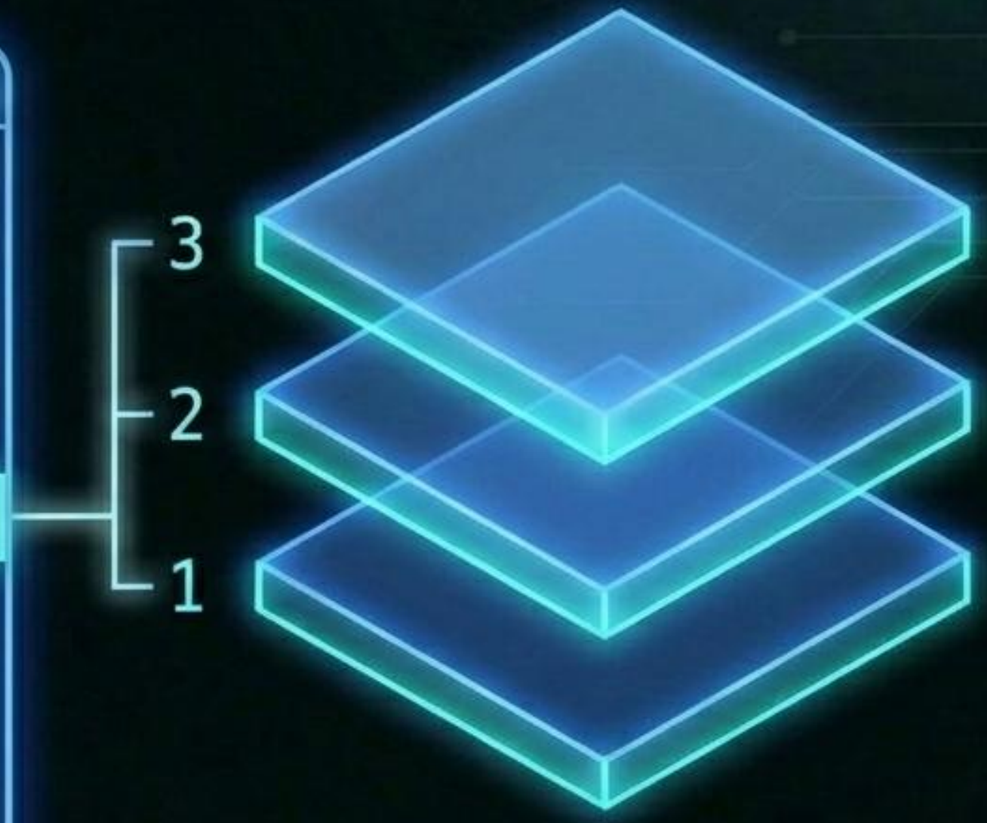
(자동)

	c1	c2
0	10	40
1	20	50
2	30	60

생성 기법 2: 인덱스명 지정

데이터 값과 행(Index) 라벨만 명시적 주입

```
df = pd.DataFrame(
    data=[[10,40],[20,50],[30,60]],
    index=[1,2,3]
)
```



결과 출력 구조 (Output Visualization 2)

(자동)

(지정)

	0	1
1	10	40
2	20	50
3	30	60

생성 기법 3: 전면 제어 (Full Control)

칼럼명, 인덱스명 동시 지정

Columns

Index

```
df = pd.DataFrame(
    data={'c1':[10,20,30], 'c2':[40,50,60]},
    index=[1,2,3]
)
```

NumPy 연동 프레임워크 (NumPy Integration)

NumPy 배열 기반 DataFrame 생성 준비

NumPy 1차 배열 변환

Step 1: NumPy 배열 생성

```
arr = np.array([[10,20,30],
                [40,50,60]])
```

Data Flow

Step 2: DataFrame 주입

```
df = pd.DataFrame(arr)
```

배열 및 다중 파라미터 동시 적용

```
arr = np.array([[10,40],
                [20,50],
                [30,60]])
```

```
index=[1,2,3]
```

```
columns=['c1','c2']
```

pd.DataFrame()
constructor

	c1	c2
1	10	40
2	20	50
3	30	60

대체 호출 문법 (Alternative Import)

클래스 직접 import 방식 적용

기본 방식

```
import pandas as pd

a = pd.DataFrame(data=[...])
```

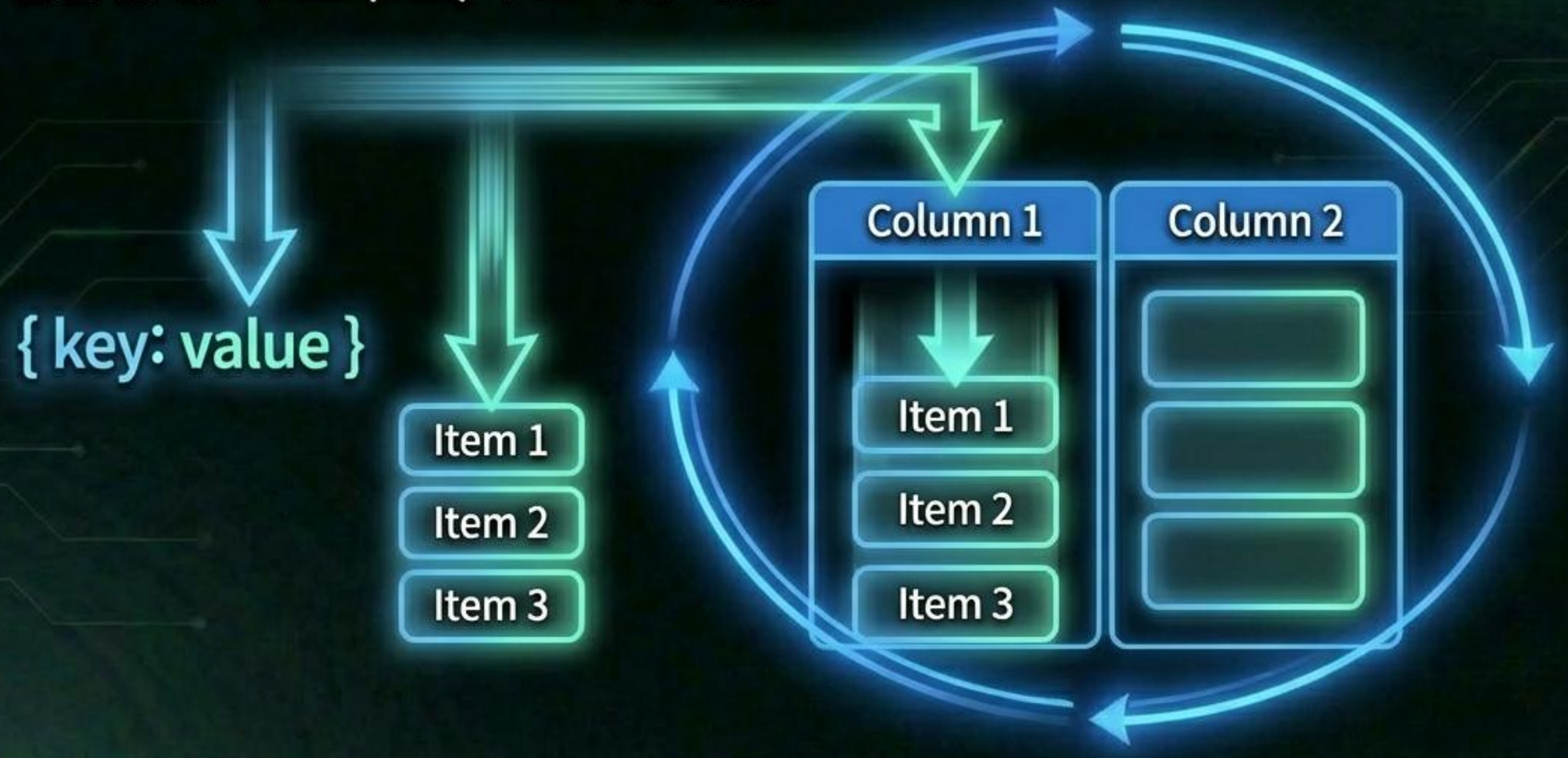
클래스 직접 호출

```
from pandas import DataFrame as df

a = df(data=[[10,40],
             [10,50],
             [30,60]])
```

딕셔너리 연동 프레임워크 (Dictionary Integration)

- Key = DataFrame의 Column으로 자동 매핑
- Value = 단일 값 및 리스트(List) 구조 적용 가능



딕셔너리 매핑 심화 실습

```
dic = {'c1':[10,20,30], 'c2':[40,50,60]}
df = pd.DataFrame(dic)
```

c1	c2
10	40
20	50
30	60

구조 변경: 행(Row) 추가 기초

- .loc[] 인덱서를 활용한 명시적 데이터 삽입

DataFrame객체명.loc[새로운 행이름] = 값 혹은 배열

행 추가 실무 적용 (Before & After)

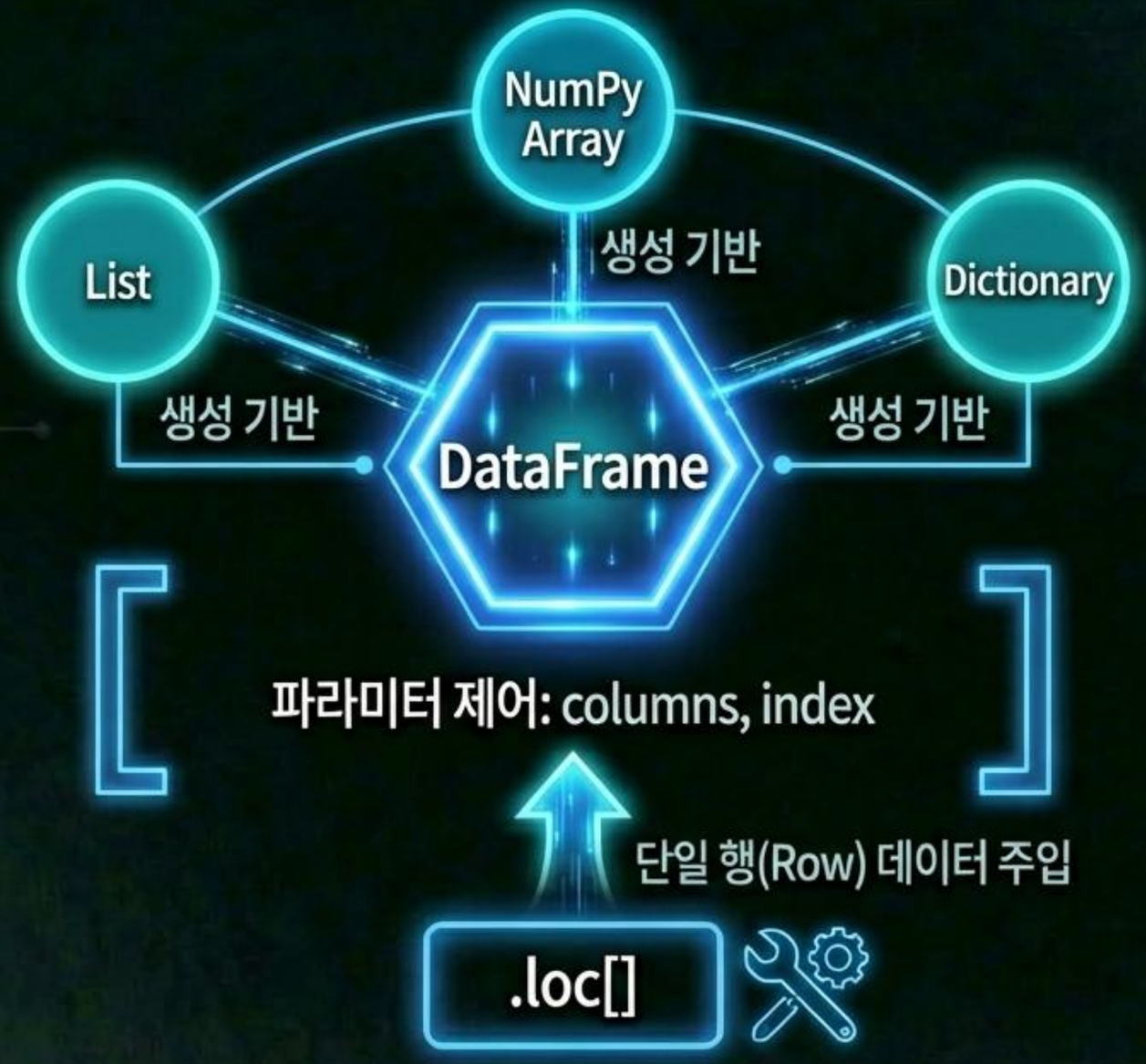
```
df = pd.DataFrame({'name':['hong','kim','heo'],'kor':[80,90,75],'eng':[80,95,100]})
```

	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100

```
df.loc[3] = ['song', 95, 80]
```

	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100
3	song	95	80

Part 1 핵심 요약 (Synthesis & Summary)



단일 열 추가 아키텍처

- **개념:** 기존 DataFrame 우측 끝에 새로운 데이터 차원(Column) 결합
- **기본 문법:** DataFrame 객체명[새로운 열이름] = 값 혹은 배열

객체명[새로운 열이름] = 값 혹은 배열

[새로운 열이름]: 사이언 텍스트 (문자열 형태)

=: 할당 연산자

값 혹은 배열: 네온 블루 텍스트 (기존 행 개수와 일치 필수)

실전 적용: 수학 점수 열 추가

- 코드 실행: `df['math'] = [70, 85, 90]`

BEFORE

	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100

AFTER

	name	kor	eng	math
0	hong	80	80	70
1	kim	90	95	85
2	heo	75	100	90

행/열 삭제: drop() 메서드

- 기본 구조: df.drop(대상, axis, inplace)
- 핵심 파라미터 해부:

방향성 제어: axis=0 vs axis=1

axis=0 (행 삭제)

- 타겟: 행 인덱스 (Row Index)
- 작동 방향: 가로(수평) 축 기준 삭제
- 예: df.drop(0, axis=0) -> 0번 행 소멸

	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100

axis=1 (열 삭제)

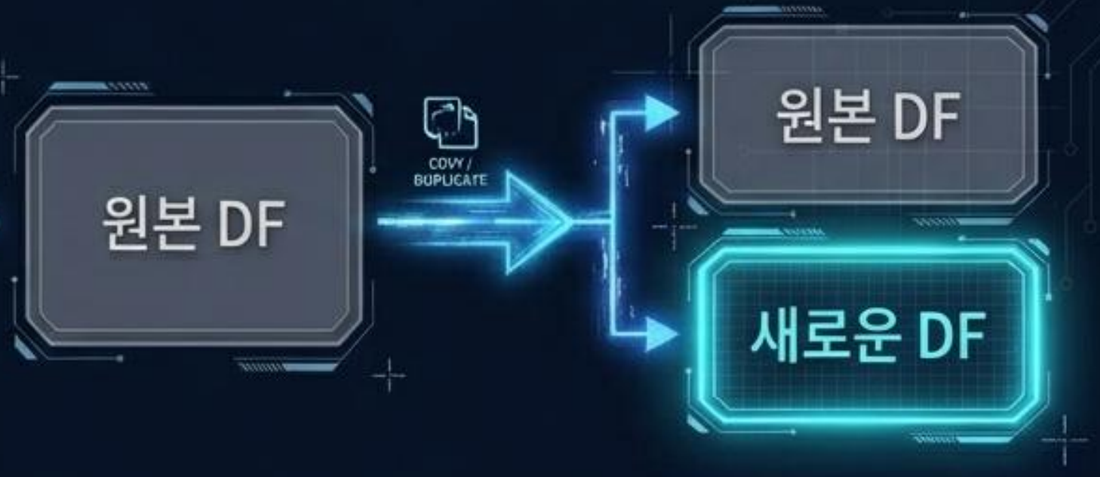
- 타겟: 열명 (Column Name)
- 작동 방향: 세로(수직) 축 기준 삭제
- 예: df.drop('eng', axis=1) -> eng 열 소멸

	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100

메모리 객체 반영 매트릭스: inplace

inplace=False (기본값)

- 작동 원리: 원본 객체 보존, 변경된 새로운 DataFrame 반환
- 사용 패턴: new_df = df.drop(...)



inplace=True

- 작동 원리: 새로운 객체 생성 없이 원본 DataFrame 자체를 즉시 변형
- 주의: 할당 불필요 (df.drop(..., inplace=True))

[Section Title] 02. 행, 열, 원소 선택하기

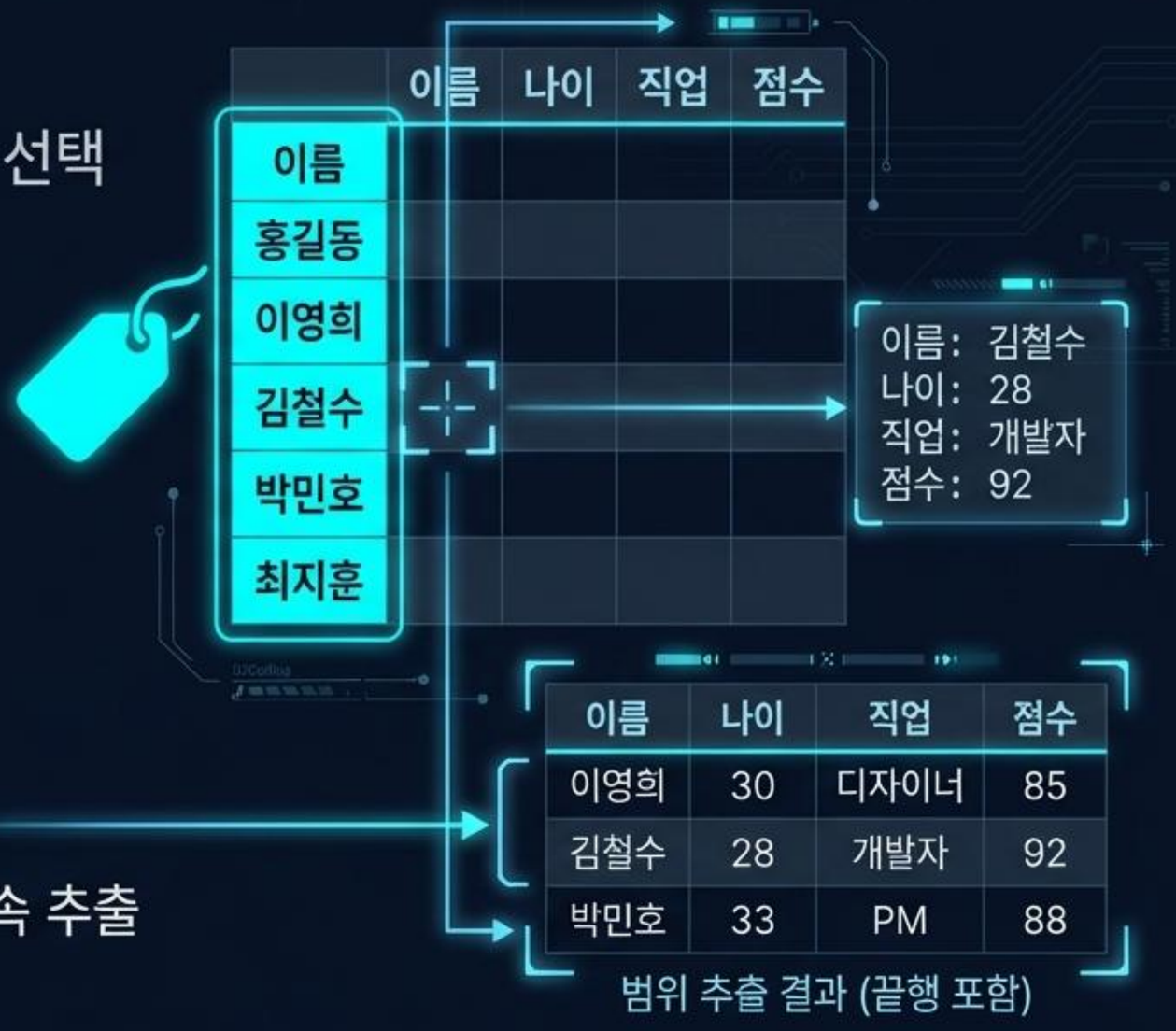
[Sub-text] loc 및 iloc를 활용한 정밀 데이터 타겟팅 추출

행 추출 1: loc 메커니즘

- 핵심 개념: '행 이름(Label)' 기반 데이터 선택

- 단일 행 선택
  - 문법: `df.loc[선택할 행이름]`
  - 결과: Series 형태로 1개 행 반환

- 범위 지정 선택
  - 문법: `df.loc[시작행이름 : 끝행이름]`
  - 특징: 시작행부터 끝행까지(끝행 포함) 연속 추출



행 추출 2: iloc 메커니즘

- **핵심 개념: '정수 인덱스(순서 번호)' 기반 데이터 선택**

- **단일 행 선택**
  - 문법: df.iloc[선택할 인덱스]

- **범위 지정 및 슬라이싱**
  - 문법: df.iloc[시작인덱스 : 끝인덱스 : 슬라이싱간격]
  - 특징: 범위의 끝 인덱스는 제외됨 (Exclusive)

iloc 슬라이싱 심화 규칙

- **간격(Step) 메커니즘**
  - 간격 생략 시: 기본값 1로 간주하여 순차 추출
- **역방향 추출**
  - 문법 적용: 간격을 -1로 지정
  - 효과: 데이터가 역순(Reverse)으로 제시됨
- **전체 추출 기법**
  - 시작/끝 인덱스 생략 시 전체 행 선택 (`df.iloc[:]`)

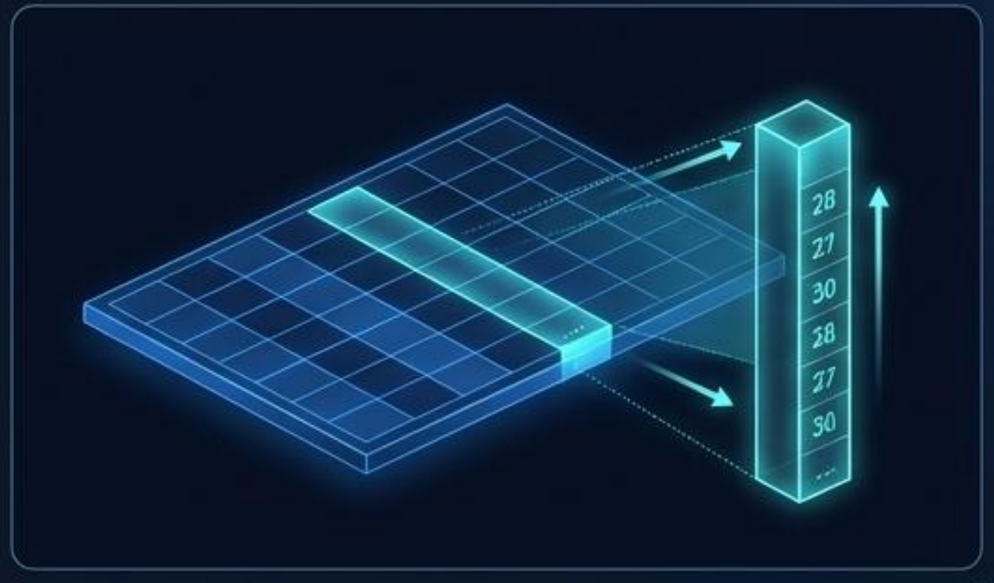
간격(Step) = 1 (순차)

간격(Step) = -1 (역방향)

열(Column) 타겟팅: 단일 vs 다중

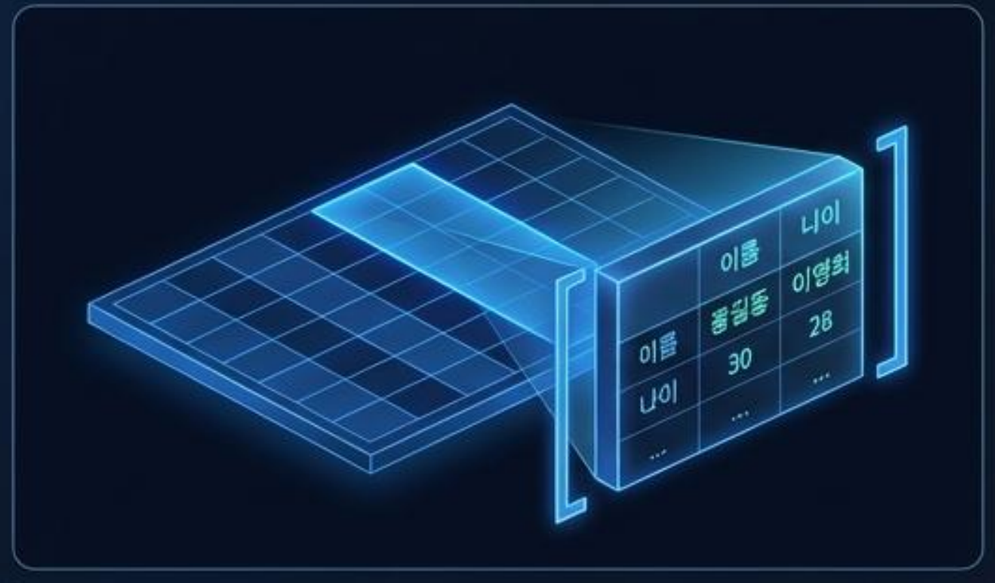
[1차원] 1개의 열 선택

- 문법: `df[선택할 열이름]`
- 구조: Series 형태로 추출



[2차원] 여러 개의 열 선택

- 문법: `df[[열이름1, 열이름2, ...]]`
- 구조: 대괄호 이중 적용 필수
- 결과: 2차원 DataFrame 형태로 추출 유지

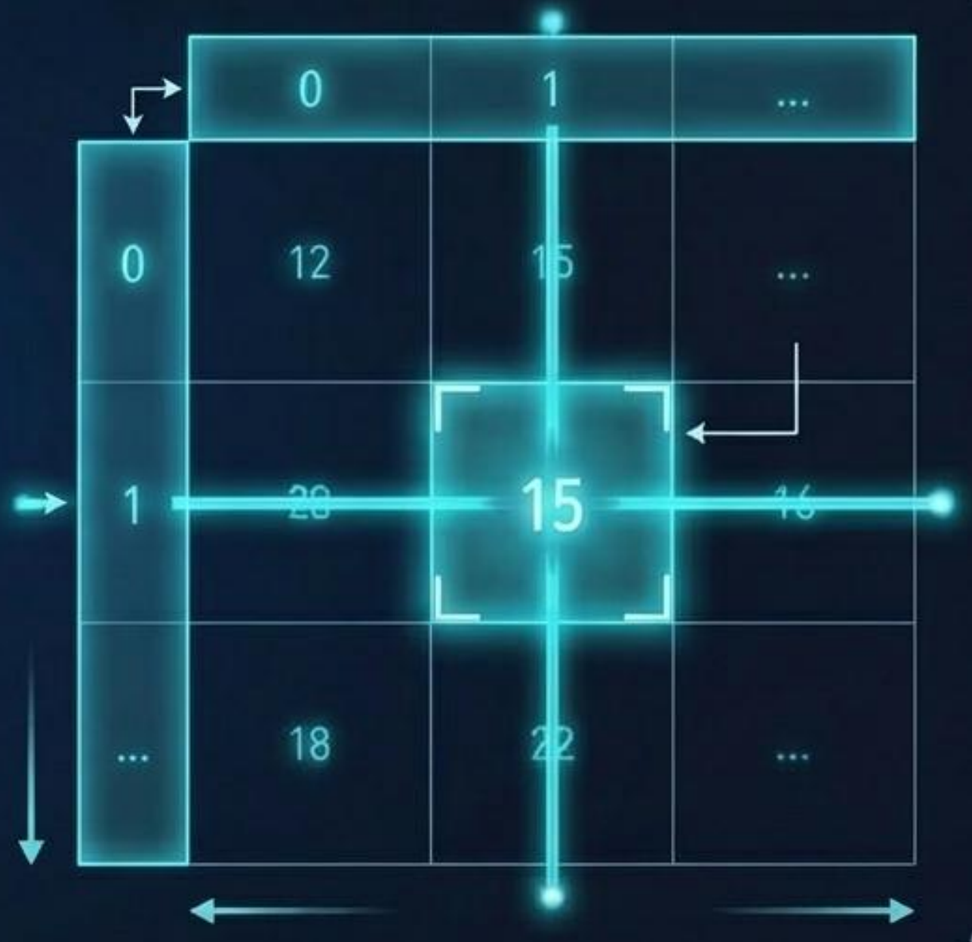


원소 정밀 조준 1: loc 교차점

- **작동 원리:** 행 이름(Label)과 열 이름(Label)의 교차 좌표 추출
- **기본 문법:**
  - `df.loc[행 인덱스/행명, 열 인덱스/열명]`
- **특징:**
  - 직관적인 이름 기반 탐색
  - 사용자 정의 라벨이 존재하는 데이터에 최적화

원소 정밀 조준 2: iloc 좌표계

- **작동 원리:** 행 번호(Integer)와 열 번호(Integer) 절대 좌표 기반 추출
- **기본 문법:**
  - `df.iloc[행 번호, 열 번호]`
- **특징:**
  - 0부터 시작하는 컴퓨터 배열 인덱싱 방식
  - 기계적/반복적 데이터 루프(Loop) 추출에 유리



loc vs iloc 코드 적용 비교

Base DataFrame Minimap

	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100

[타겟 1: 'hong'의 'eng' 점수 단일 추출]

- loc 방식: `df.loc[0, 'eng']`
- iloc 방식: `df.iloc[0, 2]`

[타겟 2: 'kim'의 'eng', 'kor' 점수 다중 추출]

- loc 방식: `df.loc[1, ['eng', 'kor']]`
- iloc 방식: `df.iloc[1, [2, 1]]`

03. 데이터 위치 변경하기

재배열 및 전치(Transpose)를 통한 구조적 뷰(View) 전환

행 위치 재조합: reindex()

- **작동 개념:** 지정된 인덱스 리스트의 순서대로 행 블록 재배치
- **사용 코드:** `df = df.reindex(index=[2, 0, 1])`
- **변환 흐름 (Before -> After):**
  - 기존 인덱스: 0, 1, 2
  - 변경 인덱스: 2, 0, 1 순으로 행 데이터 전체 셔플링

Before DataFrame Minimap

	name	kor	eng
0	'hong'	80	80
1	'kim'	90	95
2	'heo'	75	100

After DataFrame Minimap

	name	kor	eng
2	'heo'	75	100
0	'hong'	80	80
1	'kim'	90	95

열 위치 재조합: 대괄호 재나열

- **작동 개념:** 원하는 열 이름 순서대로 리스트를 구성하여 덮어쓰기
- **사용 코드:** `df = df[['name', 'kor', 'eng']]`
- **변환 흐름 (Before -> After):**
  - 기존 배열: [name, eng, kor]
  - 변경 배열: [name, kor, eng] 순으로 열 기둥 재배치

Before DataFrame Minimap

	name	eng	kor
0	'hong'	80	80
1	'kim'	95	90
2	'heo'	100	75

After DataFrame Minimap

	name	kor	eng
0	'hong'	80	80
1	'kim'	90	95
2	'heo'	75	100

축 완전 변환: 행/열 전치 (Transpose)

- **작동 개념:**
  매트릭스의 행과 열 축(Axis)을 1:1로 맞바꿈
  (90도 회전 구조)

- **명령어 구조:**
  - 정식 메서드: `df.transpose()`
  - 단축 속성: `df.T`

- **변환 결과 요약:**
  - 열(Column) 이름 -> 행(Row) 인덱스로 변환
  - 행(Row) 인덱스 -> 열(Column) 이름으로 변환

	0	1	2
name	'hong'	'kim'	'heo'
kor	80	90	75
eng	80	95	100